ACTIVITY - I
POSTMAN

Postman is designed for developers to talk directly to **APIs** (Application Programming Interfaces).

It allows you to send requests to a server, see how it responds, and test if your backend code is working correctly without needing to build a full front-end interface first.

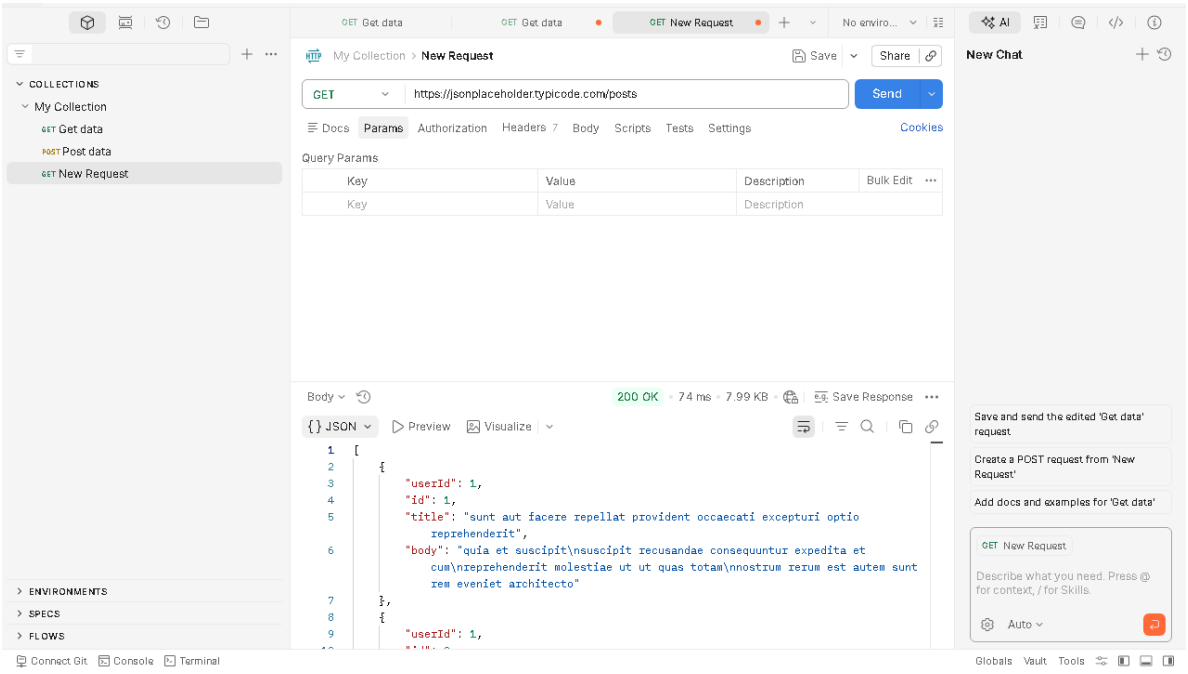
The "Big Four" HTTP Methods

In the world of APIs, we use **CRUD** (Create, Read, Update, Delete) operations. Postman helps you execute these using specific "methods."

1. GET (Read)

Used to **retrieve** data from a server. It doesn't change anything on the server; it just asks for information.

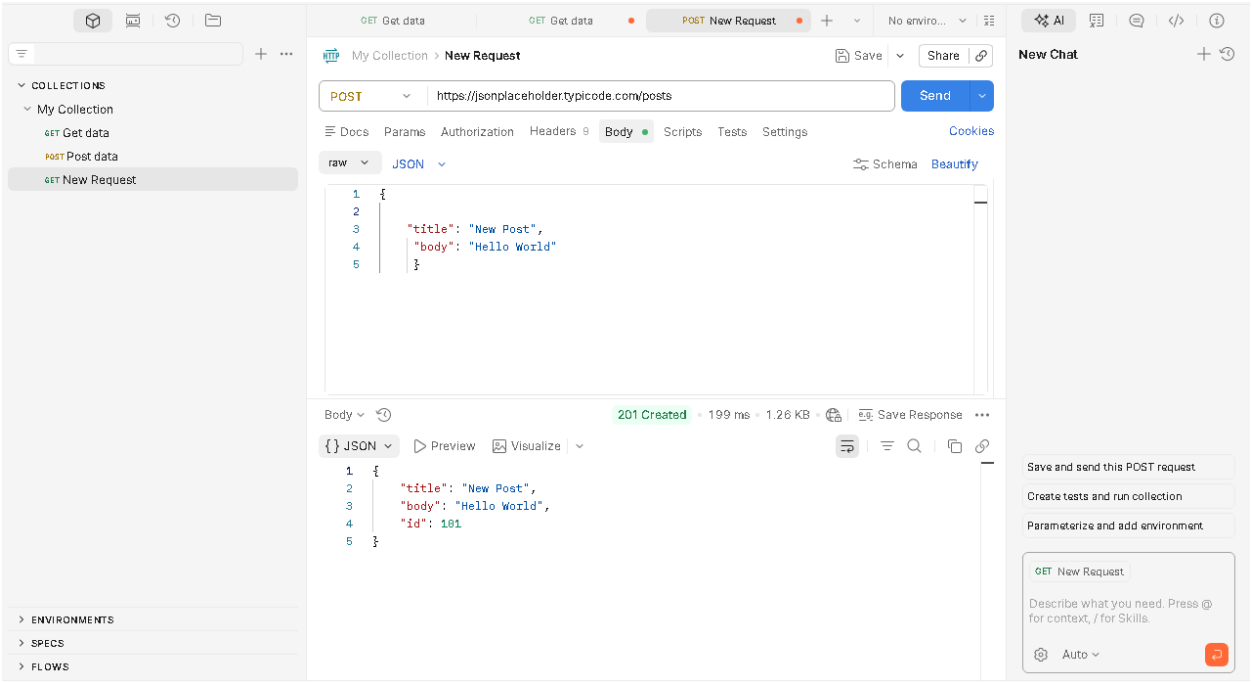
- **Real-world example:** Opening your Instagram feed to see posts.
- **Step-by-Step:**
    1. Open a new tab in Postman and select **GET** from the dropdown menu.
    2. Enter the API URL (e.g., https://jsonplaceholder.typicode.com/posts).
    3. Click **Send**.
    4. View the data in the **Response** body at the bottom.



2. POST (Create)

Used to **send** new data to a server to create a new resource.

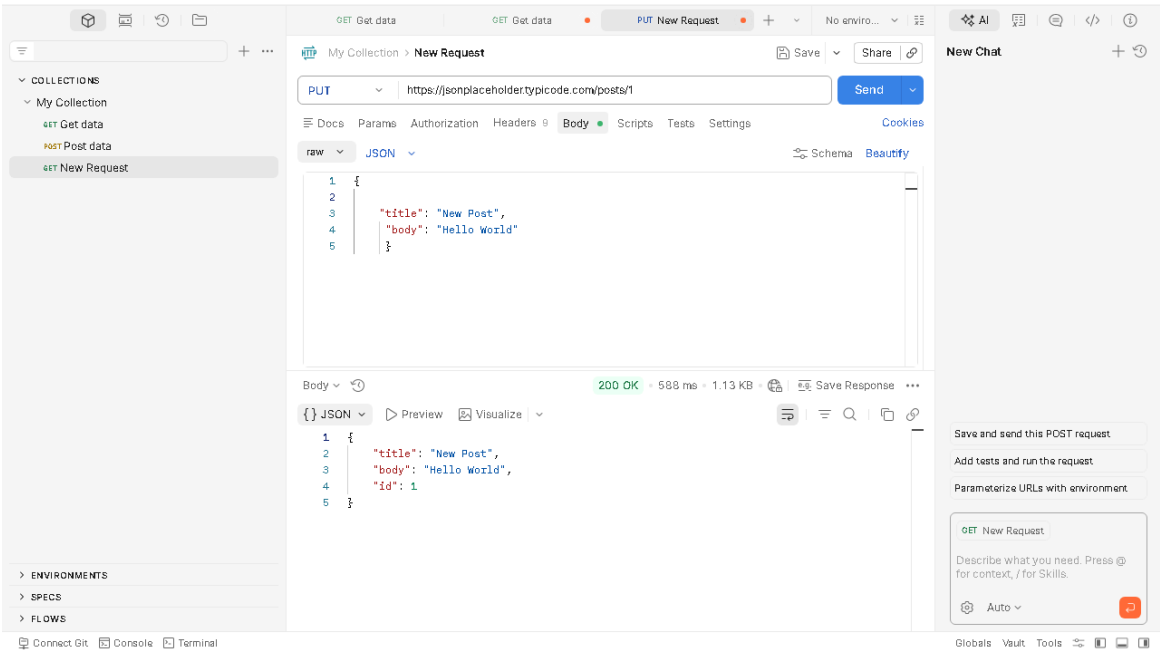
- **Real-world example:** Typing a new tweet and hitting "Post."
- **Step-by-Step:**
    1. Select **POST** from the dropdown.
    2. Enter the URL.
    3. Click the **Body** tab below the URL bar.
    4. Select **raw** and choose **JSON** from the format list.
    5. Type your data (e.g., {"title": "New Post", "body": "Hello World"}).
    6. Click **Send**.



3. PUT (Update)

Used to **update** or replace an existing resource entirely.

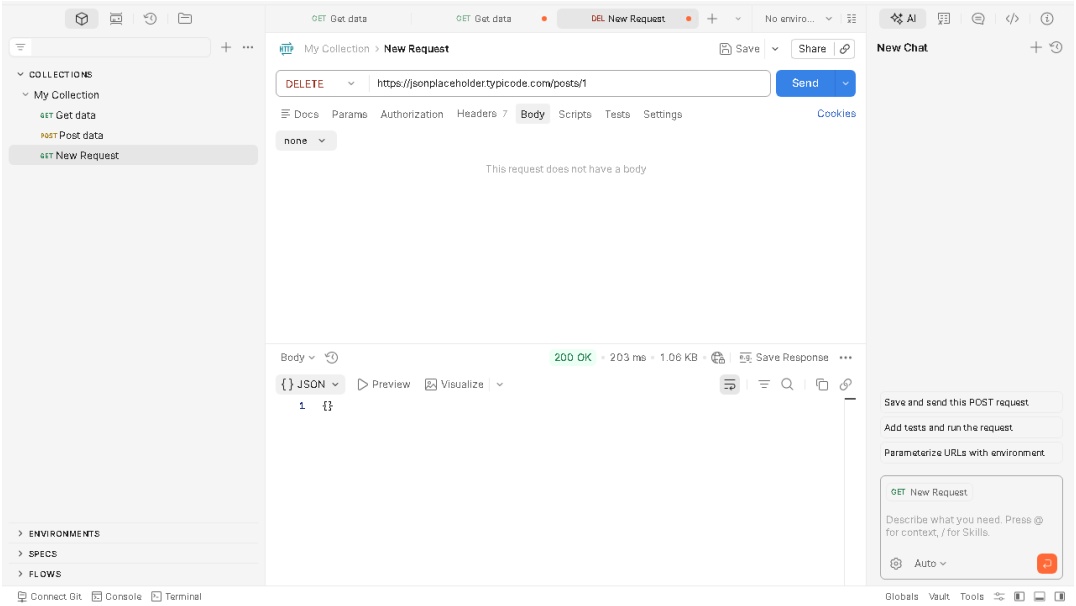
- **Real-world example:** Editing your entire profile bio and saving the changes.
- **Step-by-Step:**
    1. Select **PUT** from the dropdown.
    2. Enter the URL of the specific item (e.g., .../posts/1).
    3. Go to the **Body** tab -> **raw** -> **JSON**.
    4. Enter the updated data.
    5. Click **Send**.



4. DELETE (Remove)

Used to **delete** a specific resource from the server.

- **Real-world example:** Clicking "Delete" on a photo in your gallery.
- **Step-by-Step:**
    1. Select **DELETE** from the dropdown.
    2. Enter the URL of the specific item you want to trash (e.g., .../posts/1).
    3. Click **Send**.
    4. Look for a confirmation status (usually 200 OK or 204 No Content).



ACTIVITY - II
GITHUB COMMANDS

Core GitHub Concepts

- **Repository (Repo):** A digital folder for your project that stores all files and the history of every change made to them.
- **Commit:** A "save point" in your project's history.
- **Remote:** The version of your project stored online on GitHub's servers.

Essential GitHub Commands

Here are the most common commands used to manage your documentation and code, explained simply.

1. Setting Up

- `git init`: Turns a local folder into a brand new Git repository.
- `git clone [URL]`: Downloads an existing repository from GitHub to your computer. Use this when you want to start working on someone else's project or your own existing online project.
- **Example:** `mkdir my-project` → `cd my-project` → `git init`

2. Saving Changes

- `git add [filename]`: Prepares your file to be saved (known as "staging"). It's like putting an item in a moving box before sealing it.
- `git commit -m "your message"`: Records your changes permanently with a description. This creates a "save point" in time.

3. Syncing with GitHub

- `git push`: Sends your committed changes from your local computer up to GitHub.
- `git pull`: Grabs the latest changes from GitHub and merges them into your local computer.
  Use this to stay updated with what your teammates have written.

4. Branching (Collaboration)

- `git branch [name]`: Creates a parallel version of your project. This allows you to test new ideas without breaking the "Main" version.
- `git checkout [name]`: Switches you from one branch to another.
- `git merge [name]`: Combines the changes from a side branch back into your main project.

If you are updating a document on GitHub, your workflow usually looks like this:

1. `git pull` (Get the latest version).
2. Edit your files.
3. `git add .` (Stage all your changes).
4. `git commit -m "Fixed typo in README"` (Save the change).
5. `git push` (Upload to GitHub).

Command	Simple Meaning
Clone	Copy a project from the web to your PC.
Add	Pick which files to save.
Commit	Save changes with a note.
Push	Upload saves to the web.
Pull	Download latest saves from the web.

Status	Check which files are changed/unsaved.

ACTIVITY - III
WATERFALL, AGILE, DEVOPS

1. Waterfall (The Linear Path)

1. Waterfall Model (The "Plan-Driven" Model)

The Waterfall model is the foundation of traditional software engineering. It follows a strict **top-down** approach.

Detailed Phases:

- **Requirements Gathering:** All possible requirements of the system to be developed are captured in a document (SRS - Software Requirements Specification).
- **System Design:** Architects design the hardware and software architecture. No coding happens here.
- **Implementation:** The system is developed in small programs called units.
- **Integration & Testing:** All units are combined and tested as a whole to see if they meet the original requirements.
- **Deployment:** Once testing is done, the product is released to the customer.
- **Maintenance:** Fixing issues that arise in the client environment.

When to use it?

- When the project is short and simple.
- When requirements are very clear and fixed.
- When the environment is stable (e.g., medical device software where safety regulations are strict).

2. Agile (The Incremental Path)

Agile is not a single "method" but a philosophy based on the **Agile Manifesto**. It focuses on keeping the code functional and the customer happy.

Detailed Process:

- **Backlog:** A list of all features the customer wants.
- **Sprints:** The team picks a few items from the backlog to complete in 2 to 4 weeks.
- **Daily Stand-ups:** 15-minute daily meetings to discuss what was done yesterday and what will be done today.
- **Sprint Review:** At the end of a sprint, the team shows a "working demo" to the client.
- **Retrospective:** The team discusses how to improve their process for the next sprint.

Core Values:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

3. DevOps Model (The "Speed & Stability" Model)

DevOps is the evolution of Agile. While Agile fixed the gap between **Customers** and **Developers**, DevOps fixes the gap between **Developers** and **IT Operations**.

The "Infinity Loop" Explained:

- **Plan & Code:** Developers write code and manage versions (using GitHub).
- **Build & Test:** The code is automatically compiled and run through a battery of tests.
- **Release & Deploy:** If the tests pass, the code is automatically pushed to the live server (CI/CD).
- **Operate & Monitor:** The system is monitored for speed, errors, and user behavior. This data goes back to the "Plan" phase.

Key Practices:

- **CI (Continuous Integration):** Merging code into a central repository multiple times a day.
- **CD (Continuous Deployment):** Automatically releasing every change that passes the test suite to production.
- **IaC (Infrastructure as Code):** Managing servers and networks using configuration files rather than manual setup.

Feature	Waterfall	Agile	DevOps
Structure	Linear / Sequential	Iterative / Sprints	Continuous / Automated
Flexibility	Very Low	Very High	Extremely High
Feedback	Only at the very end	After every Sprint	Constantly (Real-time)
Goal	Finish the project	Improve the product	Speed and Reliability
Delivery	One big release	Frequent small releases	Continuous delivery
Team Focus	Siloed (Separate teams)	Collaborative (Cross-functional)	Integrated (Dev + Ops)